

Module – Class Structure, Fields, Methods, and Access Modifiers

What Is a Class?

Key Points

- A **class** defines the blueprint for creating objects.
- It combines **fields (data)** and **methods (behavior)**.
- Objects are specific instances of a class.

Code Snippet

```
class Car {  
    String color;  
    int speed;  
  
    void accelerate() {  
        speed += 10;  
        System.out.println("Speed: " + speed);  
    }  
}
```

 **Exercise:** Exercise_01_CreateClass.java

Create a class `Book` with fields `title`, `author`, and a method `displayDetails()` that prints them.

Basic Structure of a Java Class

Key Points

Typical structure includes:

1. Package declaration
2. Imports
3. Class declaration
4. Fields
5. Constructors
6. Methods

Code Snippet

```
package com.learnato.training;

public class Employee {
    private String name;
    private double salary;

    public Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }

    public void displayInfo() {
        System.out.println(name + " earns " + salary);
    }
}
```

 **Exercise:** Exercise_02_EmployeeStructure.java

Write a class `Student` with constructor and a `showProfile()` method.
Print name and grade.

Fields (Instance & Static)

Key Points

- **Instance fields:** data unique to each object.
- **Static fields:** shared across all objects.
- Access static fields using the class name.

Code Snippet

```
class Student {  
    String name;  
    static String school = "Green Valley High";  
}
```

 **Exercise:** Exercise_03_StaticVsInstance.java
Create two `Student` objects and change the `school` name.
Observe how static and instance values behave differently.

Methods (Behavior)

Key Points

- Define actions or operations of the class.
- Can access fields and take parameters.
- Use `return` to send results.

Code Snippet

```
class Calculator {  
    int add(int a, int b) {  
        return a + b;  
    }  
}
```

 **Exercise:** `Exercise_04_Methods.java`

Create a class `MathOps` with methods `add`, `subtract`, and `multiply`.
Test them in `main()`.

Static Methods and Constants

Key Points

- Static methods belong to the **class**, not the object.
- Best for **utility** or **helper** functions.

Code Snippet

```
class MathUtil {  
    static int square(int n) {  
        return n * n;  
    }  
}  
System.out.println(MathUtil.square(4)); // 16
```

 **Exercise:** Exercise_05_StaticMethods.java

Write a static method `isEven(int n)` that returns `true` if the number is even.

Access Modifiers Overview

Modifier	Same Class	Same Package	Subclass	Other Packages
public	✓	✓	✓	✓
protected	✓	✓	✓	✗
<i>default</i>	✓	✓	✗	✗
private	✓	✗	✗	✗

 **Exercise:** `Exercise_06_AccessTest.java`

Create two classes in same and different packages.

Test which fields/methods are accessible with each modifier.

public and private Members

Key Points

- `public` — accessible from anywhere.
- `private` — accessible only within the class.

Code Snippet

```
public class Account {  
    private double balance;  
  
    public void deposit(double amount) { balance += amount; }  
    public double getBalance() { return balance; }  
}
```

 **Exercise:** `Exercise_07_PrivateEncapsulation.java`

Make all fields private and provide getters/setters.

Demonstrate encapsulation.

protected and Default (Package-Private)

Key Points

- `protected` → accessible to subclasses and within the same package.
- Default (no modifier) → visible only in same package.

Code Snippet

```
class Vehicle {  
    protected int speed;  
}  
class Car extends Vehicle {  
    void accelerate() { speed += 10; }  
}
```

 **Exercise:** Exercise_08_ProtectedUsage.java

Create a superclass `Shape` with a protected field `color`.

Subclass `Circle` modifies and displays it.

Combining Modifiers in Class Design

Key Points

- Use **encapsulation**: keep data private, expose public methods.
- `protected` for extensibility, `private` for security.

Code Snippet

```
public class BankAccount {  
    private double balance;  
    protected double getBalance() { return balance; }  
    public void deposit(double amt) { balance += amt; }  
}
```

 **Exercise:** `Exercise_09_ModifierCombinations.java`

Experiment with `private`, `protected`, and `public` in one class hierarchy.
See which members are accessible where.

Nested Classes (Overview)

Key Points

- A class can be declared inside another → called **nested class**.
- Used for logically grouping helper classes or encapsulating details.

Code Snippet

```
class Outer {  
    private int data = 10;  
  
    class Inner {  
        void show() { System.out.println("Data = " + data); }  
    }  
}
```

 **Exercise:** Exercise_10_NestedClass.java

Create a nested `Logger` class inside `App` that prints messages with a timestamp.

Key Takeaways

- A **class** encapsulates state (fields) and behavior (methods).
- **Access modifiers** define visibility and encapsulation boundaries.
- Use **private fields + public methods** for secure design.
- Static members belong to the class, not instances.
- Nested classes can group logic internally and maintain modularity.

Module – static Members and Initialization Order (static vs instance)

Understanding static Members

Key Points

- `static` means the member belongs to the **class**, not any particular object.
- All objects share a single copy of a static field or method.
- Used for **constants**, **utility methods**, or **shared counters**.

Code Snippet

```
class Counter {  
    static int count = 0; // shared across all objects  
  
    Counter() { count++; }  
}  
  
Counter c1 = new Counter();  
Counter c2 = new Counter();  
System.out.println(Counter.count); // 2
```

 **Exercise:** Exercise_01_StaticCounter.java

Create a class `Student` with a **static** `totalStudents` counter that increments in the constructor. Print after creating 3 students.

Static Fields vs Instance Fields

Feature	Static Field	Instance Field
Belongs to	Class	Object
Memory	Single copy	One per object
Access	Class name or object	Object reference only
Lifetime	Exists for entire program	Exists till object is referenced

Code Snippet

```
class Person {  
    static String species = "Human";  
    String name;  
}
```

Access:

```
System.out.println(Person.species); // ✅ preferred  
System.out.println(p1.species);    // ⚠️ works, but not recommended
```

🔗 **Exercise:** Exercise_02_StaticVsInstance.java

Create Car with brand (static) and model (instance).

Check differences in output when you modify brand from one object.

Static Methods

Key Points

- Can be called without creating an object.
- Can **only access static members** directly.
- Common in utility/helper classes.

Code Snippet

```
class MathUtils {  
    static int add(int a, int b) {  
        return a + b;  
    }  
}  
System.out.println(MathUtils.add(3, 4)); // 7
```

 **Exercise:** Exercise_03_StaticMethod.java

Create a Converter class with static methods toKm(miles) and toMiles(km).

Static Initialization Block

Key Points

- Runs **once** when the class is loaded into memory (before any object is created).
- Used for initializing static variables with complex logic.

Code Snippet

```
class Config {  
    static String ENV;  
    static {  
        ENV = System.getenv("APP_ENV") != null ? "PROD" : "DEV";  
        System.out.println("Static block executed: " + ENV);  
    }  
}
```

 **Exercise:** Exercise_04_StaticBlock.java

Add print statements to observe when static block executes relative to constructor calls.

Instance Initialization Block

Key Points

- Runs every time an object is created, before the constructor.
- Used for common initialization shared across all constructors.

Code Snippet

```
class User {  
    {  
        System.out.println("Instance block: setting defaults");  
    }  
    User() {  
        System.out.println("Constructor: setting name");  
    }  
}  
new User();
```

Output:

```
Instance block: setting defaults  
Constructor: setting name
```

 **Exercise:** Exercise_05_InstanceBlock.java

Create a class with both instance block and constructor; print messages to track order.

Initialization Order (Detailed)

Key Points

Java initializes in the following order:

- 1 ☐ **Static fields & static blocks** — once, when class loads.
- 2 ☐ **Instance fields & instance blocks** — each time an object is created.
- 3 ☐ **Constructor** — last, after all blocks/fields are set.

Code Snippet

```
class Demo {  
    static { System.out.println("1. Static block"); }  
    { System.out.println("2. Instance block"); }  
    Demo() { System.out.println("3. Constructor"); }  
}  
Demo d1 = new Demo();  
Demo d2 = new Demo();
```

Output:

```
1. Static block  
2. Instance block  
3. Constructor  
2. Instance block  
3. Constructor
```

 **Exercise:** Exercise_06_InitializationOrder.java

Reproduce above and predict output for multiple object creations.

Field Initialization Order

Key Points

- Fields are initialized **in the order they appear** in the class.
- Initialization order is crucial when fields depend on each other.

Code Snippet

```
class InitOrder {  
    int a = 10;  
    int b = a + 5;  
  
    InitOrder() {  
        System.out.println("a=" + a + ", b=" + b);  
    }  
}
```

Output:

a=10, b=15

 **Exercise:** Exercise_07_FieldInitOrder.java
Swap declaration order and see if it changes output.

Static and Instance Together

Key Points

- Static blocks run once when class loads.
- Instance blocks and constructors run per object.
- Useful for tracing load vs creation events.

Code Snippet

```
class Combo {  
    static { System.out.println("Static init"); }  
    { System.out.println("Instance init"); }  
    Combo() { System.out.println("Constructor"); }  
}
```

Output:

```
Static init  
Instance init  
Constructor  
Instance init  
Constructor
```

 **Exercise:** Exercise_08_StaticInstanceMix.java

Create multiple instances of a class and print messages from all three blocks to confirm sequence.

Static Nested Classes

Key Points

- Static nested classes **do not require an instance** of the outer class.
- Useful for grouping logic logically related to the outer class but independent of its instances.

Code Snippet

```
class Outer {  
    static class Inner {  
        static void greet() { System.out.println("Hello from static nested  
class"); }  
    }  
}  
Outer.Inner.greet();
```

 **Exercise:** Exercise_09_StaticNested.java

Create a static nested class `Logger` inside `App` that logs messages with timestamp.

Common Mistakes & Best Practices

Do's:

- ✓ Use static for constants or shared utilities.
- ✓ Initialize static members using static blocks for complex setup.
- ✓ Prefer instance variables for per-object data.

Don'ts:

- ✗ Modify static variables via instance references.
- ✗ Overuse static for convenience — it breaks OOP design.
- ✗ Depend on initialization order implicitly (be explicit instead).

 **Exercise:** `Exercise_10_StaticMistakes.java`

Intentionally access a static variable via an object. Observe compiler warnings and fix using best practices.

Key Takeaways

- `static` = class-level; `instance` = object-level.
- Static members are initialized once, shared by all objects.
- Static → before instance → before constructor.
- Instance blocks run before constructors for every object.
- Use initialization blocks for setup logic; avoid hidden dependencies.

Module – Constructors (Default, Parameterized, Overloaded)

What Is a Constructor?

Key Points

- A **constructor** is a special method used to **initialize** new objects.
- Called **automatically** when an object is created using `new`.
- Has **same name as the class** and **no return type** (not even `void`).
- Used to set up default or initial values for object fields.

Code Snippet

```
class Person {
    String name;
    int age;

    Person() {
        System.out.println("Constructor called!");
    }
}
```

Usage:

```
Person p = new Person(); // invokes constructor
```

 **Exercise:** Exercise_01_SimpleConstructor.java

Create a class `Book` that prints “Book created” when a new object is created.

Default Constructor

Key Points

- If no constructor is defined, Java provides a **default constructor** automatically.
- The default constructor initializes all fields to their **default values**.
- If you define your own constructor, the compiler **does not** generate one.

Code Snippet

```
class Student {  
    String name;  
    int age;  
    // No constructor defined → compiler adds a default one  
}  
  
Student s = new Student();  
System.out.println(s.name); // null  
System.out.println(s.age);  // 0
```

 **Exercise:** Exercise_02_DefaultConstructor.java

Remove and add constructors alternately in your class to see when Java adds the default one.

Parameterized Constructor

Key Points

- Constructors can take parameters to initialize object state during creation.
- Allows flexible initialization of each object with different data.

Code Snippet

```
class Student {
    String name;
    int age;

    Student(String n, int a) {
        name = n;
        age = a;
    }
}

Student s1 = new Student("Alice", 20);
System.out.println(s1.name + " - " + s1.age);
```

 **Exercise:** Exercise_03_ParameterizedConstructor.java

Create a Car class with fields brand and price.

Use a parameterized constructor to initialize and print both values.

Constructor Overloading

Key Points

- A class can have **multiple constructors** with different parameter lists.
- This is called **constructor overloading**.
- The compiler selects the right constructor based on the **number and types** of arguments.

Code Snippet

```
class Rectangle {
    int width, height;

    Rectangle() {
        width = 1; height = 1;
    }
    Rectangle(int w, int h) {
        width = w; height = h;
    }
    Rectangle(int side) {
        width = height = side;
    }
}
```

Usage:

```
Rectangle r1 = new Rectangle();
Rectangle r2 = new Rectangle(5, 10);
Rectangle r3 = new Rectangle(4);
```

 **Exercise:** Exercise_04_ConstructorOverload.java

Create a class Employee with three constructors:

1 ☐ No-arg constructor → default values

2 ☐ One-arg → name only

3 ☐ Two-arg → name and salary.

Print data from each.

Constructor Chaining (this())

Key Points

- You can call one constructor from another in the same class using `this()`.
- Helps reduce **code duplication**.
- Must be the **first statement** inside the constructor.

Code Snippet

```
class Product {
    String name;
    double price;

    Product() {
        this("Unknown", 0.0);
    }

    Product(String n, double p) {
        name = n;
        price = p;
    }
}
```

 **Exercise:** Exercise_05_ConstructorChaining.java

Create Laptop class with a no-arg constructor calling the parameterized constructor using `this()`.

Calling Parent Constructors (super)

Key Points

- A subclass constructor can call its superclass constructor using `super()`.
- `super()` must be the **first statement** inside the subclass constructor.
- If not explicitly called, Java automatically inserts a **no-arg `super()`**.

Code Snippet

```
class Animal {
    Animal() { System.out.println("Animal created"); }
}
class Dog extends Animal {
    Dog() {
        super(); // calls Animal()
        System.out.println("Dog created");
    }
}
```

 **Exercise:** Exercise_06_SuperConstructor.java

Create a class `Person` → parent, and `Teacher` → subclass.

Use `super()` in `Teacher` to call `Person` constructor.

Initialization Order with Constructors

Key Points

- 1 ☐ Static blocks → executed once at class load.
- 2 ☐ Instance variables and blocks → executed before constructor.
- 3 ☐ Constructor → runs last when object is created.

Code Snippet

```
class Demo {  
    static { System.out.println("Static block"); }  
    { System.out.println("Instance block"); }  
    Demo() { System.out.println("Constructor"); }  
}  
new Demo();
```

Output:

```
Static block  
Instance block  
Constructor
```

 **Exercise:** Exercise_07_InitOrderConstructor.java

Add static, instance blocks, and constructors in a class to observe the sequence.

Copy Constructor (Manual)

Key Points

- Java doesn't have a built-in copy constructor (like C++).
- You can manually define one to **copy fields** from another object.

Code Snippet

```
class Point {
    int x, y;

    Point(int x, int y) { this.x = x; this.y = y; }

    // Copy constructor
    Point(Point p) {
        this.x = p.x;
        this.y = p.y;
    }
}

Point p1 = new Point(10, 20);
Point p2 = new Point(p1);
```

 **Exercise:** Exercise_08_CopyConstructor.java

Create a `Book` class with a copy constructor that duplicates another book object.

Common Mistakes

Mistake	Why It's Wrong
Declaring a constructor with return type	Becomes a normal method
Forgetting to call <code>super()</code> when needed	Compilation error if superclass has no default constructor
Doing heavy work in constructor	Slows object creation
Initializing static data inside instance constructor	Inefficient; use static blocks instead

 **Exercise:** `Exercise_09_ConstructorMistakes.java`

Deliberately write a constructor with a return type and observe compiler behavior. Then fix it.

Best Practices

✓ Do's:

- Use constructors for initialization, not computation.
- Use `this()` to avoid duplicate code in overloaded constructors.
- Use `super()` for proper superclass initialization.
- Validate constructor arguments early (throw exceptions if invalid).

✗ Don'ts:

- Don't create unnecessary constructors (let compiler provide default if enough).
- Don't perform I/O or long operations in constructors.

🔗 **Exercise:** `Exercise_10_ConstructorBestPractices.java`

Implement validation inside constructor — e.g., age cannot be negative.
Throw `IllegalArgumentException` if invalid.

Key Takeaways

- Constructors initialize object state and run automatically.
- Default → compiler-provided; Parameterized → custom setup; Overloaded → flexibility.
- `this()` chains within class, `super()` calls parent constructor.
- No return type, runs only once per object creation.
- Write clean, validating, non-heavy constructors for robust code.

Module – Object Creation and Reference Assignment Rules

Understanding Object Creation

Key Points

- Objects in Java are created dynamically using the `new` keyword.
- `new` allocates memory on the **heap**, initializes fields, and calls the **constructor**.
- Every object gets a unique memory reference.
- Reference variables store the **address** (pointer) to the object, not the object itself.

Code Snippet

```
Car c1 = new Car();    // Object created in heap
Car c2 = c1;           // Reference copy
```

Memory Diagram:

Heap: [Car@1a2b] ← c1, c2

 **Exercise:** Exercise_01_ObjectCreation.java

Create a class `Book`, make two references point to the same object, and print if `b1 == b2`.

Object Creation Lifecycle

Key Points

When `new` is used:

- 1 ☐ Memory allocated on heap.
- 2 ☐ Fields initialized with default or explicit values.
- 3 ☐ Constructors executed.
- 4 ☐ Reference assigned to variable.

Code Snippet

```
class Student {  
    String name = "Unknown";  
    Student() { System.out.println("Constructor called"); }  
}  
Student s = new Student();
```

Output:

```
Constructor called
```

 **Exercise:** Exercise_02_ObjectLifecycle.java

Add print statements in constructor and field initializations to visualize order.

Reference Variables

Key Points

- A reference variable **does not hold the object itself**, only its address.
- Multiple variables can reference the **same object**.
- Assigning one reference to another copies the **address**, not the object.

Code Snippet

```
Car c1 = new Car("Honda");  
Car c2 = c1; // both refer to same object  
c2.brand = "Toyota";  
System.out.println(c1.brand); // Toyota
```

Explanation:

c1 and c2 point to the same memory location.

 **Exercise:** Exercise_03_ReferenceSharing.java

Create an `Employee` object and two references to it.

Change data via one and print using the other to confirm shared reference.

Object Reassignment

Key Points

- When you assign a new object to an existing reference, the **old reference is lost** (if no other variable points to it).
- Such objects become **eligible for garbage collection**.

Code Snippet

```
Book b = new Book("Java");  
b = new Book("Python"); // old "Java" object lost
```

Result:

The first `Book` object is unreachable and may be garbage-collected.

 **Exercise:** `Exercise_04_Reassignment.java`

Assign a reference to multiple new objects and print the message from the constructor to see creation sequence.

Null References

Key Points

- `null` is a **special literal** that means the reference points to no object.
- Accessing members of a null reference → `NullPointerException`.
- Always check for `null` before using references.

Code Snippet

```
Car c = null;  
System.out.println(c.brand); // ❌ NullPointerException
```

Safe Access Example:

```
if (c != null) System.out.println(c.brand);
```

 **Exercise:** `Exercise_05_NullReference.java`

Create a null reference and handle it safely using an `if` check.

Comparing References

Key Points

- `==` compares **reference equality** (same memory address).
- `.equals()` compares **content equality** (overridden in many classes).

Code Snippet

```
String s1 = new String("Java");
String s2 = new String("Java");
System.out.println(s1 == s2);           // false
System.out.println(s1.equals(s2));      // true
```

 **Exercise:** `Exercise_06_ReferenceEquality.java`
Compare two `Student` objects with `==` and `.equals()`.
Override `.equals()` in your class to compare names.

Reference Type and Object Type

Key Points

- The **reference type** determines what members you can access.
- The **object type** determines which implementation runs at runtime.
- This enables **polymorphism**.

Code Snippet

```
Animal a = new Dog(); // reference type = Animal, object type = Dog
a.speak();             // calls Dog's version (runtime binding)
```

 **Exercise:** Exercise_07_RefVsObjType.java

Create superclass Shape with draw() method.

Subclass Circle overrides draw().

Assign Shape s = new Circle(); and observe runtime behavior.

Anonymous Objects

Key Points

- Objects can be created without storing their reference.
- Useful for one-time use (e.g., passing as parameters).

Code Snippet

```
new Car("Tesla").start();
```

Explanation:

A temporary object is created and used immediately.

 **Exercise:** `Exercise_08_AnonymousObject.java`

Create an anonymous object that prints “Temporary object used!” in its constructor.

Garbage Collection Basics

Key Points

- When no references point to an object, it becomes **eligible for garbage collection**.
- The garbage collector reclaims memory automatically.
- You can suggest GC using `System.gc()`, but it's **not guaranteed**.

Code Snippet

```
Book b = new Book("Java");  
b = null; // eligible for GC  
System.gc(); // may trigger GC
```

Optional Method:

```
@Override  
protected void finalize() {  
    System.out.println("Book destroyed");  
}
```

 **Exercise:** `Exercise_09_GarbageCollection.java`

Create a class with `finalize()` method.

Make several objects unreachable and observe the order of destruction (may vary).

Object Aliasing and Side Effects

Key Points

- Object **aliasing** occurs when multiple references point to the same object.
- Mutating the object through one reference affects all others.
- Use defensive copies if mutation must be avoided.

Code Snippet

```
Person p1 = new Person("Alice");
Person p2 = p1;
p2.name = "Bob";
System.out.println(p1.name); // Bob
```

 **Exercise:** Exercise_10_ObjectAliasing.java

Demonstrate aliasing with two references and then fix it by cloning or copying the object.

Best Practices

Do's

- Always initialize references before use.
- Set unused references to `null` (optional, helps GC).
- Use `.equals()` for content comparison.
- Understand difference between reference and object type.
- Create copies (via constructor or `clone()`) to avoid unintended side effects.

Don'ts

- Don't access members through null references.
- Don't assume `==` compares content.
- Don't depend on `System.gc()` for deterministic cleanup.

 **Exercise:** `Exercise_11_BestPractices.java`

Write code demonstrating both bad and corrected reference handling practices.

Key Takeaways

- Objects live in heap; references store memory addresses.
- Multiple references can point to one object.
- `==` compares references; `.equals()` compares content.
- Reassigning or nullifying references makes objects eligible for GC.
- Understanding reference semantics prevents bugs and memory leaks.

Module – Variable Scope and Shadowing

What Is Variable Scope?

Key Points

- **Scope** defines where a variable can be accessed in a program.
- Each variable exists only **within the block** where it is declared.
- Blocks are defined by { } — class, method, or loop boundaries.
- Outside its scope, a variable is **invisible** and **cannot be used**.

Code Snippet

```
void demoScope() {  
    int x = 10; // visible inside this method  
    if (true) {  
        int y = 20; // visible only inside if-block  
        System.out.println(x + y);  
    }  
    // System.out.println(y); ❌ Error: y not in scope  
}
```

 **Exercise:** Exercise_01_BasicScope.java

Create a method with nested blocks and try accessing variables declared inside/outside each block.

Types of Variable Scope in Java

Type	Declared Inside	Visible In	Lifetime
Local Variable	Method or block	Within that method/block	Until method returns
Instance Variable	Inside class, outside methods	Within all instance methods	Until object is garbage-collected
Static Variable	Inside class, marked <code>static</code>	Accessible via class name	Until JVM exits

Code Snippet

```
class ScopeExample {
    static int globalCount = 0;    // static
    int id;                       // instance
    void increment() {
        int temp = 1;             // local
        id += temp;
        globalCount++;
    }
}
```

 **Exercise:** `Exercise_02_ScopeTypes.java`

Create a class with a static counter, an instance field, and a local variable. Observe their visibility and lifetime.

Local Variables

Key Points

- Declared inside methods, constructors, or blocks.
- Must be **explicitly initialized** before use.
- Automatically destroyed when method exits.

Code Snippet

```
void test() {  
    int a; // not initialized  
    a = 5;  
    System.out.println(a);  
}
```

 **Exercise:** Exercise_03_LocalVars.java

Try to print a local variable before initializing it — note the compiler error.

Instance and Static Variables

Key Points

- **Instance variables** → tied to each object.
- **Static variables** → shared among all instances.
- Both are initialized automatically (default values).

Code Snippet

```
class Account {  
    static int totalAccounts; // class-level  
    double balance;          // object-level  
}
```

 **Exercise:** Exercise_04_InstanceStaticScope.java

Print static and instance fields from multiple objects to see shared vs. independent behavior.

Block Scope

Key Points

- Any variable declared within {} is **block-scoped**.
- Variables created inside loops or conditions are destroyed after the block ends.

Code Snippet

```
for (int i = 1; i <= 3; i++) {  
    int square = i * i;  
    System.out.println(square);  
}  
// System.out.println(square); ❌ Out of scope
```

 **Exercise:** Exercise_05_BlockScope.java

Create nested loops and print the visibility of `i` and `j` inside/outside their blocks.

Variable Shadowing – Concept

Key Points

- **Shadowing** occurs when a variable in an inner scope **hides** a variable with the same name in an outer scope.
- The inner variable temporarily **“shadows”** the outer one within its scope.
- To access the outer variable, use `this` (for instance variables) or the class name (for static).

Code Snippet

```
class ShadowDemo {  
    int x = 10;  
    void show() {  
        int x = 20; // shadows instance variable  
        System.out.println("Local x = " + x);  
        System.out.println("Instance x = " + this.x);  
    }  
}
```

 **Exercise:** `Exercise_06_ShadowingBasics.java`

Create a class where a local variable shadows an instance field.

Use `this` to access both values.

Shadowing in Nested Blocks

Key Points

- Shadowing can happen at **any inner block** level.
- The inner declaration overrides the outer one only within that block.

Code Snippet

```
int num = 5;
{
    int num = 10; // shadows outer 'num'
    System.out.println(num); // 10
}
System.out.println(num); // 5
```

 **Exercise:** Exercise_07_NestedShadowing.java

Write a program with a variable declared at method level and redeclared in a nested block; print both.

Constructor and Parameter Shadowing

Key Points

- Parameters can shadow instance fields with the same name.
- Use `this.fieldName` to distinguish instance fields from parameters.

Code Snippet

```
class Student {  
    String name;  
    Student(String name) {  
        this.name = name; // assigns parameter to instance field  
    }  
}
```

 **Exercise:** Exercise_08_ParameterShadowing.java

Remove `this` and see how it breaks initialization.

Then re-add it to fix the issue.

Static Field Shadowing

Key Points

- A subclass can **declare a static field** with the same name as in the superclass.
- It **hides**, not overrides, the parent's static field.
- Access depends on the **reference type**, not object type.

Code Snippet

```
class A { static int val = 5; }  
class B extends A { static int val = 10; }  
  
A obj = new B();  
System.out.println(obj.val); // 5 – determined by reference type A  
System.out.println(B.val);   // 10
```

 **Exercise:** Exercise_09_StaticShadowing.java

Create superclass–subclass pair with same static variable and observe output via different references.

Shadowing vs. Hiding vs. Overriding

Concept	Applies To	Resolved At	Behavior
Shadowing	Local hides outer variable	Compile-time	Within nested scopes
Hiding	Static field in subclass hides parent's	Compile-time	No polymorphism
Overriding	Instance method in subclass replaces parent's	Runtime	Supports polymorphism

 **Exercise:** `Exercise_10_CompareShadowHideOverride.java`

Create one example each for shadowing, hiding, and overriding; run and observe differences.

Common Pitfalls

- ✗ Declaring variables with the same name in nested scopes — causes confusion.
- ✗ Forgetting `this` in constructors → shadowed parameters remain unassigned.
- ✗ Accessing out-of-scope variables → compile-time error.

Code Snippet

```
class Account {  
    double balance;  
    Account(double balance) {  
        balance = balance; // ✗ doesn't assign to field  
    }  
}
```

✓ Fixed:

```
this.balance = balance;
```

 **Exercise:** `Exercise_11_ShadowingPitfall.java`

Fix a constructor where parameter shadowing prevented initialization.

Best Practices

Do's

- ✓ Use clear and distinct variable names to avoid shadowing.
- ✓ Use `this` explicitly when parameters overlap with field names.
- ✓ Limit variable scope as much as possible for clarity and memory efficiency.
- ✓ Follow consistent naming conventions (e.g., instance fields start with lowercase nouns).

Don'ts

- ✗ Declare variables with the same name in nested scopes.
- ✗ Access or rely on out-of-scope variables.
- ✗ Shadow class-level variables unintentionally.

 **Exercise:** `Exercise_12_BestPractices.java`

Refactor a messy code snippet with shadowed and poorly scoped variables to make it clean and readable.

Key Takeaways

- Scope determines where a variable lives and can be used.
- Variables declared inside blocks disappear when the block ends.
- Shadowing temporarily hides outer variables in nested scopes.
- Use `this` to differentiate instance variables from parameters.
- Keep scope small, names clear, and avoid shadowing for readable, bug-free code.

Module – Object Lifecycle & Garbage Collection Basics

Understanding Object Lifecycle

Key Points

- Every object in Java goes through **creation**, **use**, and **destruction** phases.
- Object lifecycle is managed **automatically by the JVM** — developers cannot explicitly destroy objects.
- Key stages:
 - 1 ☐ **Object creation** (_{new})
 - 2 ☐ **Active use** (referenced by variables)
 - 3 ☐ **Dereferencing / becoming unreachable**
 - 4 ☐ **Garbage collection & memory reclamation**

Code Snippet

```
Student s = new Student("Asha"); // created
s = null;                        // dereferenced → eligible for GC
```

 **Exercise:** Exercise_01_ObjectLifecycle.java

Create an object, use it, then set its reference to `null` and print a message before/after to track its lifecycle.

Object Creation (Heap Allocation)

Key Points

- All objects in Java are stored in the **heap memory**.
- The `new` keyword triggers:
 - Memory allocation in heap
 - Initialization of instance fields
 - Invocation of the constructor
 - Object reference (address) is stored on the **stack**.

Code Snippet

```
Car c = new Car("Tesla"); // heap allocation + constructor call
```

Diagram:

```
Stack: c → [Car@1234]  
Heap:  [Car@1234: brand="Tesla"]
```

 **Exercise:** Exercise_02_ObjectCreation.java

Visualize object creation using print statements inside a constructor.

Object Reachability

Key Points

- An object is **reachable** as long as at least one live reference points to it.
- If no references exist → **unreachable** → **eligible for GC**.
- Cyclic references between objects are also collectible if no external reference exists.

Code Snippet

```
class Node {  
    Node next;  
}  
Node a = new Node();  
Node b = new Node();  
a.next = b;  
b.next = a;  
a = null;  
b = null; // both eligible for GC
```

 **Exercise:** Exercise_03_Reachability.java
Create circularly linked objects and dereference them.
Draw how GC still cleans them up.

Eligible vs Collected Objects

Key Points

- An object becomes **eligible** for GC when no active references exist.
- Collection is **not immediate** — JVM decides when to run GC.
- Eligible $\neq$ Collected $\rightarrow$ object may still exist temporarily in heap.

Code Snippet

```
Book b = new Book("Java");  
b = null; // eligible for GC, not yet collected  
System.gc(); // suggest GC (not guaranteed)
```

 **Exercise:** Exercise_04_Eligibility.java

Print a message from the constructor and later from `finalize()` to observe eligibility vs collection timing.

The `finalize()` Method (Legacy)

Key Points

- `finalize()` was historically used to perform cleanup before object destruction.
- Called **once before GC reclaims the object**.
- Now **deprecated** in Java 9+ — replaced by `Cleaner` or `try-with-resources`.

Code Snippet

```
class Temp {
    @Override
    protected void finalize() {
        System.out.println("Finalize called!");
    }
}

Temp t = new Temp();
t = null;
System.gc(); // may trigger finalize
```

 **Exercise:** `Exercise_05_Finalize.java`

Implement `finalize()` in a small class and observe if/when it executes after `System.gc()`.

Modern Cleanup: AutoCloseable & try-with-resources

Key Points

- Instead of relying on `finalize()`, Java uses **deterministic cleanup** via `try-with-resources`.
- Classes implementing `AutoCloseable` or `Closeable` close automatically after use.

Code Snippet

```
try (BufferedReader br = new BufferedReader(new FileReader("data.txt"))) {  
    System.out.println(br.readLine());  
} catch (IOException e) {  
    e.printStackTrace();  
}
```

 **Exercise:** `Exercise_06_TryWithResources.java`

Create a custom `AutoCloseable` class that prints messages when opened and closed.
Use it in a `try-with-resources` block.

Garbage Collection Mechanism

Key Points

- JVM uses **Garbage Collector (GC)** to reclaim memory of unreachable objects.
- Types of GC algorithms (JVM-dependent):
 - **Serial GC** – simple, single-threaded (small apps).
 - **Parallel GC** – uses multiple threads for speed.
 - **G1 (Garbage First) GC** – default modern collector, balances pause time.
 - GC is **automatic** and runs when memory pressure occurs.

Code Snippet

```
System.gc(); // requests GC (may or may not run)
Runtime.getRuntime().gc(); // another way
```

 **Exercise:** Exercise_07_GCAlgorithms.java

Research the default GC in your JVM using:

```
java -XX:+PrintCommandLineFlags -version
```

and note which collector is used.

Generational Heap Model

Key Points

- Java heap is divided into **generations** for efficient GC:
 - **Young Generation** – new objects; frequent minor GCs.
 - **Old (Tenured) Generation** – long-lived objects.
 - **Metaspace** – class metadata and static info.
 - Short-lived objects are collected quickly; long-lived ones are promoted.

Diagram:

```
[Heap Memory]
├── Young Gen → Eden + Survivor spaces
├── Old Gen → Long-lived objects
└── Metaspace → Class info
```

 **Exercise:** Exercise_08_GenerationalGC.java

Create many short-lived objects in a loop and monitor memory via VisualVM or jconsole.

Reference Types (Strong, Weak, Soft, Phantom)

Key Points

- Not all references prevent GC:
 - **Strong Reference:** Normal variable → prevents GC.
 - **Weak Reference:** Collected when no strong references remain.
 - **Soft Reference:** Retained until memory is low.
 - **Phantom Reference:** Used for post-GC cleanup tracking.

Code Snippet

```
WeakReference<Book> ref = new WeakReference<>(new Book("GC"));  
System.out.println(ref.get()); // may become null after GC
```

 **Exercise:** Exercise_09_WeakReference.java

Create a weak reference to an object, call `System.gc()`, and check whether it still exists.

Common Misconceptions

- ✗ “Calling `System.gc()` destroys all objects immediately.” → GC is **only a request**.
- ✗ “`finalize()` guarantees cleanup.” → Deprecated, unreliable.
- ✗ “Memory leaks can’t happen in Java.” → Possible if references are held unnecessarily (e.g., static collections).
- ✗ “Once GC runs, program pauses completely.” → Only briefly, depends on GC type.

 **Exercise:** `Exercise_10_GCMyths.java`

Write examples that demonstrate these misconceptions and verify actual behavior.

Best Practices for Object Lifecycle Management

- ✓ Keep references **short-lived** when possible.
- ✓ Set unused references to `null` in long-running loops.
- ✓ Prefer **try-with-resources** for deterministic cleanup.
- ✓ Avoid large static collections holding objects indefinitely.
- ✓ Use **profiling tools** (VisualVM, JConsole, JMC) to analyze memory leaks.

 **Exercise:** `Exercise_11_BestPractices.java`

Simulate a small program with and without clearing references in a loop — observe heap usage difference.

Key Takeaways

- Objects are created on heap and destroyed automatically by JVM GC.
- `finalize()` is deprecated — use `AutoCloseable` instead.
- Objects become GC-eligible once all references are gone.
- GC algorithms optimize memory and performance behind the scenes.
- Understand lifecycle → write cleaner, memory-efficient Java programs.

Module – Encapsulation, Immutability, and Defensive Copies

What Is Encapsulation?

Key Points

- **Encapsulation** = bundling of data (fields) and behavior (methods) inside a class.
- It hides internal implementation and exposes only necessary operations.
- Aim: **data protection, controlled access, and clean API design.**
- Achieved using **access modifiers** (`private`, `public`, etc.) and **getters/setters**.

Code Snippet

```
class Account {  
    private double balance; // hidden data  
  
    public void deposit(double amount) {  
        if (amount > 0) balance += amount;  
    }  
  
    public double getBalance() {  
        return balance;  
    }  
}
```

 **Exercise:** `Exercise_01_EncapsulationBasics.java`

Make a class `Student` with **private** fields (`name`, `grade`) and provide public methods to access/update them safely.

Why Encapsulation Matters

Key Points

- ✅ Prevents unauthorized or accidental modification of data.
- ✅ Improves maintainability (changes inside class don't affect other code).
- ✅ Supports validation and controlled updates.
- ✅ Enables thread-safety and consistency.

Bad Example:

```
public class Account {  
    public double balance; // ❌ anyone can modify  
}
```

Better:

```
private double balance; // ✅ restricted access  
public void setBalance(double amount) {  
    if (amount >= 0) balance = amount;  
}
```

 **Exercise:** Exercise_02_DataHiding.java

Refactor a class with public fields to use encapsulation with validation logic.

Getters, Setters, and Validation

Key Points

- Use **getters** to read private fields.
- Use **setters** to modify them safely.
- Add validation logic inside setters to ensure valid state.

Code Snippet

```
class Person {  
    private int age;  
  
    public void setAge(int age) {  
        if (age >= 0 && age <= 120)  
            this.age = age;  
        else  
            throw new IllegalArgumentException("Invalid age");  
    }  
  
    public int getAge() {  
        return age;  
    }  
}
```

 **Exercise:** Exercise_03_GettersSetters.java

Add validation to ensure name isn't null or blank when set via setter.

Concept of Immutability

Key Points

- An **immutable object** cannot change its state once created.
- All fields are set in the constructor and never modified later.
- Immutability simplifies debugging and makes code thread-safe.
- Common examples: `String`, wrapper classes (`Integer`, `Double`, etc.).

Code Snippet

```
final class Coordinate {  
    private final int x, y;  
  
    Coordinate(int x, int y) {  
        this.x = x; this.y = y;  
    }  
  
    int getX() { return x; }  
    int getY() { return y; }  
}
```

 **Exercise:** `Exercise_04_ImmutableClass.java`

Create an immutable class `Point` with `x` and `y`.

Verify you can't modify fields after construction.

Rules for Making a Class Immutable

- 1 ☐ Declare the class **final** → prevents subclass modification.
- 2 ☐ Make all fields **private and final**.
- 3 ☐ Initialize fields only in the constructor.
- 4 ☐ Provide **no setters**.
- 5 ☐ Avoid exposing mutable internal objects directly (use **defensive copies**).

Code Snippet

```
final class Person {
    private final String name;
    private final Date birthDate; // mutable type!

    Person(String name, Date birthDate) {
        this.name = name;
        this.birthDate = new Date(birthDate.getTime()); // defensive copy
    }

    public String getName() { return name; }

    public Date getBirthDate() {
        return new Date(birthDate.getTime()); // defensive copy
    }
}
```

 **Exercise:** Exercise_05_ImmutableRules.java

Implement an immutable **Employee** class that has **name**, **salary**, and a **Date** field.
Ensure defensive copying in both constructor and getter.

Defensive Copies (Concept)

Key Points

- A **defensive copy** prevents external code from modifying internal mutable fields.
- Used when:
 - Accepting mutable objects as constructor arguments.
 - Returning internal mutable objects from getters.
 - Instead of exposing the original, return a **new copy**.

Example Without Defensive Copy:

```
class User {  
    private Date created;  
    User(Date created) { this.created = created; }  
    public Date getCreated() { return created; } // ❌ dangerous  
}
```

External Modification:

```
Date d = new Date();  
User u = new User(d);  
d.setYear(2020); // changes internal state!
```

Fixed:

```
public Date getCreated() {  
    return new Date(created.getTime()); // ✅ defensive copy  
}
```

🔗 **Exercise:** Exercise_06_DefensiveCopy.java

Demonstrate how a mutable `Date` leaks internal state, then fix using a defensive copy.

Immutable Collections

Key Points

- Use factory methods or wrappers to create **unmodifiable collections**.
- Prevents external modification of internal data structures.

Code Snippet

```
import java.util.*;

class Library {
    private final List<String> books;

    Library(List<String> books) {
        this.books = List.copyOf(books); // creates unmodifiable copy
    }

    public List<String> getBooks() {
        return List.copyOf(books);
    }
}
```

 **Exercise:** Exercise_07_ImmutableCollection.java

Create a **Team** class that stores players in a **List**.

Ensure the list returned by `getPlayers()` can't be modified externally.

Encapsulation vs Immutability

Aspect	Encapsulation	Immutability
Purpose	Hide internal data	Prevent data change
Access	Controlled via getters/setters	Only via getters (no setters)
State	Mutable but protected	Permanent and fixed
Typical Use	Entities, models	Value objects, constants

 **Exercise:** `Exercise_08_EncapVsImmutable.java`

Refactor an encapsulated class into an immutable version and compare differences in design.

Benefits of Immutability

- ✓ Simpler debugging — state never changes.
- ✓ Thread-safe — no synchronization needed.
- ✓ Safe for caching, hashing, and sharing.
- ✓ Helps build predictable, functional-style code.

Example:

```
String s1 = "Java";  
String s2 = s1.concat("21");  
System.out.println(s1); // Java  
System.out.println(s2); // Java21
```

 **Exercise:** Exercise_09_ImmutableBehavior.java

Modify a `String` and observe that a new object is created each time.

Common Mistakes

- ✗ Forgetting to copy mutable inputs in constructor.
- ✗ Returning internal mutable objects directly.
- ✗ Declaring class as immutable but leaving setters.
- ✗ Using `final` on references to mutable objects without copies.

Code Snippet (Wrong):

```
final class Employee {  
    private final Date hireDate;  
    public Date getHireDate() { return hireDate; } // ✗ unsafe  
}
```

 **Exercise:** `Exercise_10_ImmutableMistakes.java`

Write and fix a class that claims to be immutable but leaks its internal mutable state.

Best Practices

- ✓ Favor **immutable** objects for data transfer and configuration classes.
- ✓ Always create **defensive copies** for mutable inputs/outputs.
- ✓ Combine **encapsulation** and **immutability** for strong design.
- ✓ Use `record` types (Java 16+) for simple immutable data carriers.

Code Snippet (Using `record`):

```
public record Point(int x, int y) {}
```

 **Exercise:** `Exercise_11_RecordImmutable.java`

Rewrite your `Coordinate` class as a Java `record` and compare simplicity.

Key Takeaways

- Encapsulation hides data and enforces controlled access.
- Immutability ensures data cannot change after creation.
- Defensive copies protect internal mutable state from leaks.
- Combine all three for safe, predictable, and thread-safe class design.
- Java `record` and immutable collections simplify implementation.

Module – Using this and super in Constructors and Methods

Understanding `this` Keyword

Key Points

- `this` is a **reference to the current object**.
- Used when a **local variable or parameter name shadows** an instance variable.
- Also used to call another constructor within the same class.
- Can be passed as an argument to another method or constructor.

Code Snippet

```
class Student {  
    String name;  
    Student(String name) {  
        this.name = name; // distinguishes field from parameter  
    }  
  
    void display() {  
        System.out.println("Student: " + this.name);  
    }  
}
```

 **Exercise:** Exercise_01_ThisKeyword.java

Write a class with a field `city`. Add a constructor parameter with the same name and use `this` to correctly assign it.

Calling Methods with `this`

Key Points

- `this` allows one instance method to call another in the same class.
- Often optional (compiler adds it automatically), but helps clarity.
- Useful when passing the current object as an argument.

Code Snippet

```
class Employee {  
    void work() {  
        System.out.println("Working...");  
    }  
  
    void report() {  
        this.work(); // same as just work();  
        System.out.println("Reporting complete.");  
    }  
}
```

 **Exercise:** Exercise_02_ThisInMethod.java

Call one method from another using `this` and print the call sequence.

`this()` Constructor Call

Key Points

- `this()` calls **another constructor** within the same class.
- Helps avoid code duplication across multiple constructors.
- Must be the **first statement** inside the constructor.

Code Snippet

```
class Car {
    String brand;
    int year;

    Car() {
        this("Unknown", 2025); // calls another constructor
    }

    Car(String brand, int year) {
        this.brand = brand;
        this.year = year;
    }
}
```

 **Exercise:** Exercise_03_ThisConstructor.java

Create overloaded constructors in a class `Book`.

Use `this()` in the no-arg constructor to call the parameterized one.

Understanding `super` Keyword

Key Points

- `super` refers to the **immediate parent class**.
- Used to access parent's:
 - **Fields** (if hidden by subclass).
 - **Methods** (if overridden).
 - **Constructor** (via `super()` — first statement in subclass constructor).

Code Snippet

```
class Animal {  
    void sound() { System.out.println("Animal sound"); }  
}  
class Dog extends Animal {  
    void sound() {  
        super.sound(); // call parent method  
        System.out.println("Bark");  
    }  
}
```

 **Exercise:** Exercise_04_SuperKeyword.java

Override a method in subclass and use `super` to call parent's version before the subclass logic.

Using `super()` in Constructors

Key Points

- `super()` calls a **parent constructor**.
- Must be the **first statement** inside subclass constructor.
- If not specified, the compiler inserts a **no-arg `super()`** automatically.

Code Snippet

```
class Vehicle {
    Vehicle(String type) {
        System.out.println("Vehicle type: " + type);
    }
}

class Car extends Vehicle {
    Car() {
        super("Car"); // calls parent constructor
        System.out.println("Car created");
    }
}
```

 **Exercise:** Exercise_05_SuperConstructor.java

Create parent class `Person` and subclass `Teacher`.

Call parent constructor using `super (name)`.

Accessing Hidden Fields with `super`

Key Points

- If a subclass declares a field with the same name as the parent, the parent's field is **hidden**.
- Use `super.fieldName` to access the parent version.

Code Snippet

```
class Parent {
    int value = 10;
}
class Child extends Parent {
    int value = 20;
    void show() {
        System.out.println("Parent value: " + super.value);
        System.out.println("Child value: " + value);
    }
}
```

 **Exercise:** Exercise_06_SuperField.java

Create two classes with same field name and use `super` to print both parent and child values.

Method Overriding and `super`

Key Points

- When a method in subclass overrides a parent method, both methods exist.
- `super.methodName()` explicitly calls parent's version.
- Useful for extending parent logic rather than replacing it completely.

Code Snippet

```
class Shape {  
    void draw() { System.out.println("Drawing shape"); }  
}  
class Circle extends Shape {  
    void draw() {  
        super.draw();  
        System.out.println("Drawing circle");  
    }  
}
```

 **Exercise:** Exercise_07_SuperMethod.java

Override a method and use `super.method()` to call both versions in sequence.

this VS super Comparison

Feature	this	super
Refers To	Current object	Parent object
Used In	Current class	Subclass
Calls	Another constructor or method in same class	Parent's constructor/method
Access	Own fields/methods	Parent's fields/methods
Must Appear First In Constructor	✅ <code>this()</code>	✅ <code>super()</code> (only one can appear)

 **Exercise:** `Exercise_08_ThisVsSuper.java`

Create two classes (Base, Derived).

In constructor and methods, use both `this` and `super` appropriately and observe sequence of execution.

Execution Order in Inheritance Hierarchy

Key Points

- Constructors execute **from parent to child**.
- `super()` is always called (explicitly or implicitly).
- Instance initializers run before constructors.

Code Snippet

```
class A {  
    A() { System.out.println("A's constructor"); }  
}  
class B extends A {  
    B() {  
        super();  
        System.out.println("B's constructor");  
    }  
}  
new B();
```

Output:

```
A's constructor  
B's constructor
```

 **Exercise:** `Exercise_09_ConstructorOrder.java`

Create a 3-level class hierarchy ($A \rightarrow B \rightarrow C$) and print constructor execution order.

Using `this` and `super` in Method Context

Key Points

- Inside methods, `this` → current instance, `super` → parent context.
- `this` can be used to pass the current object to another class.
- `super` allows reuse of overridden methods to extend functionality.

Code Snippet

```
class Logger {
    void log(Employee e) {
        System.out.println("Logging: " + e);
    }
}
class Employee {
    void send(Logger logger) {
        logger.log(this); // passing current object
    }
}
```

 **Exercise:** Exercise_10_ThisInMethodParam.java

Create a `Printer` class with a `print(Employee e)` method and use `this` to pass current object.

Common Mistakes

- ✗ Calling `this()` or `super()` **not as first line** in constructor.
- ✗ Using both `this()` and `super()` in same constructor → compilation error.
- ✗ Forgetting to chain parent constructor → may cause uninitialized state.
- ✗ Accessing overridden members in constructors → risky (child not yet initialized).

Code Snippet (Incorrect):

```
class Demo {  
    Demo() {  
        System.out.println("Before this()"); // ✗ cannot precede this()  
        this(10);  
    }  
    Demo(int x) {}  
}
```

 **Exercise:** `Exercise_11_CommonMistakes.java`

Write examples where `this()` and `super()` are misused, then fix them to compile successfully.

Best Practices

- ✓ Use `this` to eliminate ambiguity between fields and parameters.
- ✓ Use constructor chaining with `this()` to centralize initialization logic.
- ✓ Always call `super()` explicitly when the parent has parameterized constructors.
- ✓ Avoid using `super` in constructors to call overridden methods (child not ready yet).
- ✓ Use clear comments when `this` or `super` affects behavior.

 **Exercise:** `Exercise_12_BestPractices.java`

Refactor a class hierarchy to use constructor chaining correctly and eliminate duplication.

Key Takeaways

- `this` → current object reference, `super` → parent object reference.
- `this()` and `super()` must be first in constructors and cannot coexist there.
- Constructors run from parent to child, ensuring full initialization.
- Use `super` to reuse parent functionality instead of rewriting it.
- Mastering `this` and `super` improves clarity, code reuse, and design consistency.

Module – Records in Java 21: Declaration, Compact Constructors, Auto-Generated Methods

What Are Records?

Key Points

- Introduced in **Java 16** (standard), enhanced in **Java 21**.
- Records are **special classes** for immutable, data-carrying objects.
- Reduce boilerplate — automatically provide:
 - `private final` fields
 - `public constructor`
 - `equals()`, `hashCode()`, and `toString()` implementations
 - Ideal for **DTOs**, **configuration objects**, or **value carriers**.

Code Snippet

```
public record Point(int x, int y) {}
```

Equivalent to:

```
final class Point {  
    private final int x, y;  
    public Point(int x, int y) { this.x = x; this.y = y; }  
    public int x() { return x; }  
    public int y() { return y; }  
    public boolean equals(Object o) {...}  
    public int hashCode() {...}  
    public String toString() {...}  
}
```

 **Exercise:** Exercise_01_RecordBasics.java

Define a simple `Book` record with `title`, `author`, and `price`.

Print its `toString()` and check if two equal records compare true with `.equals()`.

Record Declaration Syntax

Key Points

- Declared using the `record` keyword instead of `class`.
- The **header** defines the record's components (fields).
- Automatically creates **accessor methods**, **constructor**, and **canonical implementations**.

Code Snippet

```
public record Employee(String name, int id, double salary) {}
```

Accessors:

```
Employee e = new Employee("Alice", 101, 75000);  
System.out.println(e.name());    // accessor, not getName()  
System.out.println(e.salary());  // 75000
```

 **Exercise:** Exercise_02_RecordDeclaration.java

Create an `Employee` record and access its fields using accessor methods (no getters).

Auto-Generated Members

Key Points

For every record, the compiler auto-generates:

- ✓ **Constructor** → assigns all components.
- ✓ **Accessors** → one per component.
- ✓ **equals() & hashCode()** → based on all components.
- ✓ **toString()** → auto-formatted string output.

Code Snippet

```
record Product(String name, double price) {}  
Product p1 = new Product("Pen", 10);  
Product p2 = new Product("Pen", 10);  
System.out.println(p1.equals(p2)); // true  
System.out.println(p1);           // Product[name=Pen, price=10.0]
```

 **Exercise:** Exercise_03_RecordMethods.java

Create two identical records and compare using `equals()` and `hashCode()`.
Print the result of `toString()`.

Compact Constructors

Key Points

- Records allow a **compact constructor** — no parameter list needed.
- Used for **validation or transformation** before assignment.
- Compiler automatically assigns validated parameters to fields after the block runs.

Code Snippet

```
public record Account(String name, double balance) {  
    public Account {  
        if (balance < 0)  
            throw new IllegalArgumentException("Balance cannot be  
negative");  
    }  
}
```

Explanation:

✅ No need to assign `this.name = name;` → done automatically.

🔗 **Exercise:** `Exercise_04_CompactConstructor.java`

Create a `Student` record with a compact constructor that throws an exception if marks < 0 or > 100.

Canonical vs Compact Constructors

Type	Syntax	Use Case
Canonical Constructor	<code>public RecordName(Type field1, Type field2) { ... }</code>	When you need full control of assignments.
Compact Constructor	<code>public RecordName { ... }</code>	For simple validation or normalization .

Code Snippet (Canonical):

```
public record Range(int start, int end) {  
    public Range(int start, int end) {  
        if (start > end)  
            throw new IllegalArgumentException("Invalid range");  
        this.start = start;  
        this.end = end;  
    }  
}
```

 **Exercise:** `Exercise_05_CanonicalConstructor.java`
Implement a canonical constructor that swaps `start` and `end` if they're reversed.

Custom Methods in Records

Key Points

- Records can have **custom methods** like any normal class.
- Cannot add mutable fields; methods should respect immutability.

Code Snippet

```
public record Rectangle(double width, double height) {  
    public double area() {  
        return width * height;  
    }  
}
```

 **Exercise:** Exercise_06_RecordMethods.java

Add a method `area()` in a `Circle` record to compute the area from its radius.

Static Members in Records

Key Points

- Records can include **static fields**, **static methods**, and **static initializers**.
- Useful for constants or factory methods.

Code Snippet

```
public record Currency(String code) {  
    public static final Currency USD = new Currency("USD");  
    public static Currency of(String code) {  
        return new Currency(code.toUpperCase());  
    }  
}
```

 **Exercise:** Exercise_07_StaticMembers.java

Create a `Country` record with a static constant `INDIA` and a static method `of(String name)`.

Restrictions on Records

Key Points

- ❌ Cannot extend another class (implicitly extends `java.lang.Record`).
- ✅ Can **implement interfaces**.
- ❌ All fields are implicitly `private final`.
- ✅ No setters allowed.
- ✅ Inner classes allowed, but not record inheritance.

Code Snippet

```
record Vehicle(String type) implements Comparable<Vehicle> {  
    public int compareTo(Vehicle v) {  
        return this.type.compareTo(v.type);  
    }  
}
```

🔗 **Exercise:** `Exercise_08_RecordInterfaces.java`
Implement `Comparable` in a record to sort based on a field value.

Nested Records & Records in Sealed Classes (Java 21)

Key Points

- Records can be **nested** inside classes or other records.
- Can be used within **sealed class hierarchies** for restricted polymorphism.

Code Snippet

```
sealed interface Shape permits Circle, Rectangle {}  
record Circle(double radius) implements Shape {}  
record Rectangle(double width, double height) implements Shape {}
```

 **Exercise:** Exercise_09_SealedRecord.java

Create a sealed interface `Shape` and implement it with two record types: `Square` and `Circle`.

Records vs Classes

Feature	Regular Class	Record
Mutability	Mutable by default	Immutable
Boilerplate	High	Very low
Inheritance	Can extend other classes	Cannot extend classes
<code>equals()</code> / <code>hashCode()</code>	Must be written manually	Auto-generated
Use Case	Business logic	Data transfer/value objects

 **Exercise:** `Exercise_10_RecordVsClass.java`

Write both a regular `Person` class and a `Person` record with the same fields.
Compare verbosity and immutability.

Best Practices for Using Records

- ✓ Use records for **data carrier objects** (immutable DTOs, configuration, responses).
- ✓ Avoid adding mutable collections or references — prefer immutable or copies.
- ✓ Use **compact constructors** for validation only.
- ✓ Don't overload with business logic — keep records simple.
- ✓ Use meaningful, descriptive component names — they become accessor names.

🔗 **Exercise:** `Exercise_11_BestPractices.java`

Create a `LoginRequest` record with `username` and `password`.

Add compact constructor validation for empty values and test immutability.

Key Takeaways

- Records provide concise, immutable data carriers with auto-generated methods.
- Compact constructors simplify validation logic.
- Immutable by design → safer, cleaner, thread-safe code.
- Ideal for DTOs, API responses, or value objects.
- Java 21 extends records for sealed types and pattern matching.

Module – Limitations of Records & Copy/Equality Behavior

Why Records Have Limitations

Key Points

- Records were designed for **data encapsulation**, not for inheritance or mutation.
- Their **simplicity and immutability** come with trade-offs:
 - No subclassing (records are implicitly `final`).
 - All fields are `private` and `final`.
 - State cannot be changed after creation.
 - Auto-generated methods rely solely on declared components.

Code Snippet

```
public record User(String name, int age) {}  
// User cannot be extended  
// class Admin extends User {} ❌ Compilation error
```

 **Exercise:** `Exercise_01_RecordLimitations.java`

Try extending a record and adding a mutable field — observe the compiler errors and understand why they occur.

Immutability in Records

Key Points

- Records are **shallowly immutable** — you cannot reassign fields, but referenced objects may still mutate.
- If a component is a mutable type (e.g., `List`, `Date`), internal data can still be modified indirectly.
- Defensive copies are needed for **true immutability**.

Code Snippet (Unsafe):

```
record Student(String name, List<String> subjects) {}

List<String> subs = new ArrayList<>(List.of("Math", "CS"));
Student s = new Student("Asha", subs);
subs.add("Physics");
System.out.println(s.subjects()); // ["Math", "CS", "Physics"] ❌ changed!
```

Fixed (Defensive Copy):

```
record Student(String name, List<String> subjects) {
    public Student {
        subjects = List.copyOf(subjects); // defensive copy
    }
}
```

 **Exercise:** `Exercise_02_RecordImmutability.java`

Create a record with a mutable list field and show how to make it truly immutable with `List.copyOf()`.

Record Inheritance Limitations

Key Points

- Records **cannot extend** any class — they implicitly extend `java.lang.Record`.
- However, they **can implement interfaces**.
- This means you cannot create specialized subtypes of a record class.
- Use **composition or sealed interfaces** instead of inheritance.

Code Snippet

```
record Employee(String name) {}
```

```
// ❌ Not allowed:  
// class Manager extends Employee {}
```

```
interface Payable { double salary(); }  
record Staff(String name, double salary) implements Payable {}
```

 **Exercise:** `Exercise_03_RecordInheritance.java`

Create a sealed interface `Shape` and implement it using two records (`Circle`, `Square`) instead of subclassing a base record.

No Custom Mutability

Key Points

- All fields in records are **final** — no setters allowed.
- You can't reassign record fields after object creation.
- If mutability is required, use a **regular class** instead.

Code Snippet

```
record BankAccount(String name, double balance) {}  
// account.balance = 5000; ❌ not allowed
```

 **Exercise:** Exercise_04_NoMutability.java

Try to modify record fields after creation and then rewrite as a mutable class to compare behavior.

Copy Semantics in Records

Key Points

- Java records **do not provide a built-in copy constructor** or cloning method.
- To create a copy, you must explicitly construct a new record.
- If record contains **mutable components**, copies are **shallow** by default.

Code Snippet

```
record Course(String title, List<String> topics) {}

List<String> list = new ArrayList<>(List.of("OOP", "Java"));
Course c1 = new Course("CS101", list);
Course c2 = new Course(c1.title(), c1.topics()); // shallow copy

c1.topics().add("DSA");
System.out.println(c2.topics()); // reflects the change ❌
```

 **Exercise:** Exercise_05_ShallowCopy.java

Create two record instances sharing a mutable field and show how a shallow copy reflects changes.

Deep Copy Concept

Key Points

- A **deep copy** duplicates not just the record, but also its mutable components.
- Must manually clone or copy all mutable objects.
- Ensures that changes in one object do not affect the other.

Code Snippet (Deep Copy):

```
record Course(String title, List<String> topics) {  
    public Course(Course other) {  
        this(other.title(), new ArrayList<>(other.topics()));  
    }  
}
```

Usage:

```
Course c1 = new Course("CS101", List.of("Java", "OOP"));  
Course c2 = new Course(c1);  
System.out.println(c1 == c2); // false
```

 **Exercise:** Exercise_06_DeepCopy.java

Implement a deep-copy constructor for a record with a mutable list field.

Equality Behavior in Records

Key Points

- `equals()` and `hashCode()` are **auto-generated** and **component-based**.
- Two records are equal if all components are equal (using `.equals()` for each).
- Equality is **value-based**, not reference-based.

Code Snippet

```
record Point(int x, int y) {}

Point p1 = new Point(1, 2);
Point p2 = new Point(1, 2);

System.out.println(p1.equals(p2)); // true
System.out.println(p1 == p2);      // false
```

 **Exercise:** `Exercise_07_RecordEquality.java`

Create two identical records and test equality with both `==` and `.equals()`.

Shallow vs Deep Equality

Type	Description	Behavior
Shallow Equality	Compares references of nested objects	May fail if nested objects differ in memory address
Deep Equality	Compares content recursively	Must be manually implemented
Records Default	Uses shallow equality for nested mutable objects	Works fine for immutable components

Code Snippet:

```
record Person(String name, List<String> hobbies) {}

Person p1 = new Person("Ravi", new ArrayList<>(List.of("Cricket")));
Person p2 = new Person("Ravi", new ArrayList<>(List.of("Cricket")));

System.out.println(p1.equals(p2)); // false ❌ different List references
```

 **Exercise:** Exercise_08_DeepEquality.java

Create a record with a list field. Compare two records with identical contents and check equality behavior.

Practical Alternatives

Key Points

If record limitations affect your use case:

✅ Use **classes** when you need:

- Mutable fields
- Inheritance or polymorphic hierarchy
- Custom lifecycle or logic

✅ Use **records** when you need:

- Immutable data containers
- Automatic `equals()` and `hashCode()`
- Compact syntax and clean data carriers

Code Snippet:

```
// Use class for mutable scenario
class MutableUser {
    String name;
    void setName(String name) { this.name = name; }
}
```

🔗 **Exercise:** `Exercise_09_RecordVsClass.java`

Convert a record to a mutable class. Compare boilerplate and flexibility differences.

Best Practices

- ✓ Always ensure record components are **immutable or defensively copied**.
- ✓ Use records for **value-based equality**, not for mutable entities.
- ✓ Avoid sharing mutable references between records.
- ✓ Implement deep-copy constructors if mutable fields are unavoidable.
- ✓ Use classes if you require inheritance or state changes.

 **Exercise:** `Exercise_10_BestPractices.java`

Create a record with a `List` field. Add defensive copy in constructor and verify immutability after modification attempts.

Key Takeaways

- Records are **immutable**, **final**, and **value-based**.
- They **cannot** be extended or mutated after construction.
- Copies are **shallow** unless explicitly made deep.
- Equality compares all components — deep only if nested objects implement deep equality.
- Use **defensive copies** and **immutable collections** for data safety.
- If you need inheritance or mutability, use regular classes instead.